This article has been accepted and published on J-STAGE in advance of copyediting. Content is final as presented.

IEICE Electronics Express, Vol.VV, No.NN, 1–6

LETTER

A Multimode SHA-3 Accelerator based on RISC-V system

Huu-Thuan Huynh[1], Tan-Phat Dang[1a], Tuan-Kiet Tran[1], Trong-Thuc Hoang[2], and Cong-Kha Pham[2]

Abstract Nowadays, almost all fields cannot lack security, from the essential encryption/decryption to hash function algorithms. The Secure Hash Algorithm 3 (SHA-3) with four modes, SHA3-224/256/384/512, is a known new hash function due to being more secure than its predecessors, SHA-1 and SHA-2. While hardware implementations of SHA-3 have been extensively studied, the primary focus has often been on optimizing the KECCAK algorithm. This paper introduces an efficient multimode SHA-3 architecture (MS3) featuring configurable buffers and a sub-pipeline KECCAK design. These innovations aim to save resources and boost throughput, respectively. Furthermore, MS3 is integrated with the reduced instruction set computer five (RISC-V) system as a hardware accelerator via the TileLink bus. This integration enables MS3 to communicate with RISC-V for configuration purposes and utilize direct memory access (DMA) for efficient data transfer with memory. Experimental results on the Cyclone IV E platform demonstrate MS3 achieving approximately 500 Mbps throughput across all modes, with DMA achieving a throughput of 540.21 Mbps. Additionally, our design exhibits superior efficiency compared to existing works on Virtex 5, 6, and 7 FPGA platforms. Specifically, MS3 achieves throughputs of 11.07 Gbps, 14.52 Gbps, and 17.29 Gbps, with corresponding efficiencies of 10.31 Mbps/Slice, 15.03 Mbps/Slice, and 18.39 Mbps/Slice, respectively.

key words: SHA-3, KECCAK, RISC-V, Hardware Accelerator
Classification: Devices, circuits and hardware for IoT and biomedical applications

1. Introduction

A hash function generates a fixed-size hash value from an arbitrary input size. Even a slight change in the input results in a completely different hash value. With this characteristic, the hash function is widely utilized across various domains today, including data integrity [1], authentication [1, 2], digital signatures[3], random number generation [1, 4], password storage [5], and blockchain networks [6]. Given the diverse requirements for hash functions, numerous algorithms have been developed over time, including SHA-1, SHA-2, and SHA-3. In 2015, the National Institute of Standards and Technology (NIST) officially released SHA-3 [7] as a successor to SHA-1 and SHA-2, which were found to have vulnerabilities exposed by various research studies [8, 9]. Unlike SHA-1 and SHA-2, which rely on the Merkle-Damgard (MD) construction [10], SHA-3 is based on the SPONGE construction [7]. The introduction of SHA-3, with its unique SPONGE construction, aims to enhance cryptographic security and address potential vulnerabilities identified in SHA-1.

A hardware implementation of SHA-3 has been explored in various studies [11, 12, 13, 14, 15, 16, 17, 18, 19, 20]. Some studies [11, 12] emphasize resource savings by employing the folding mechanism, which allows for the reuse of resources multiple times. Another aspect of improving the throughput is explored in [13, 14, 15, 16, 20]. They propose unrolling with multiple factors to reduce the number of loops and improve throughput efficiently for single data. However, this approach may increase resource usage and decrease frequency due to longer critical paths. To address the issue of long critical paths, studies [16, 17] propose the sub-pipeline approach. This method inserts registers between step mappings in the round function (Rnd), effectively increasing frequency efficiency but doubling the number of clocks required. Some studies, such as [18, 19], combine unrolling and pipelining at the Rnd level to enhance throughput. However, this approach often leads to a significant increase in resource utilization.

The combination of the SHA-3 algorithm and RISC-V has been explored in some studies, ranging from instruction extension [21, 22] to the hardware accelerator [23], aiming to improve the execution time of the SHA-3 algorithm. Hardware accelerators are deemed more efficient due to their ability to parallelize processes, utilize the memory hierarchy effectively, and reduce fetch and decode overhead in the central processing unit (CPU) [24].

This paper proposes a new efficient MS3 architecture supporting multiple modes (SHA3-224/256/384/512) to accommodate diverse application requirements. Each mode requires unique input and output lengths. To optimize efficiently for the number of buffers for input and output, we introduce configurable buffers capable of functioning as either input or output buffers based on the selected mode of operation. Furthermore, we introduce a sub-pipeline architecture for Rnd to enhance throughput by increasing frequency thanks to shortening the critical path. Finally, MS3 is integrated into the RISC-V system as a hardware accelerator. We employ the DMA technique within the system to maximize the utilization of MS3's capabilities. The operation of the MS3 begins with the RISC-V core

[1]University of Science, Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam
[2]University of Electro-Communications (UEC), Tokyo, Japan
a) dtphat@hcmus.edu.vn

DOI: 10.1587/elex.21.20240156
Received March 08, 2024
Accepted April 16, 2024
Publicized April 30, 2024

Copyright © 2024 The Institute of Electronics, Information and Communication Engineers

configuring it, followed by initiating DMA read operations to continuously fetch data from memory, thereby maintaining high throughput. Subsequently, the generated hash value is transferred back to memory using DMA write operations.
The paper's remaining sections are structured as follows: Section 2 presents the theoretical overview of the SHA-3 algorithm. Section 3 describes the proposed hardware architecture, outlining each component. Section 4 wraps up the paper by summarizing, comparing, and assessing the results. Finally, Section 5 offers concluding remarks and outlines future development directions.

2. Background Knowledge

SHA-3 comprises four modes (SHA3-224/256/384/512), each mode corresponds to specific output and the input sizes determined by the d and r index, respectively, as outlined in Table I. All SHA-3 functions operate within a common framework known as the SPONGE construction [7], as shown in Fig. 1. This construction involves three main steps: padding, absorbing, and squeezing phase. It is highly adaptable and allows for generating hash values with variable lengths, making it well-suited for diverse applications.
Padding ensures that the message's input size becomes a multiple of the rate r by appending the sample "10*1" [7].
In the absorbing phase, the padded message is divided into multiple blocks, each of size r bits. To establish the 1600-bit state of the SPONGE construction, each block of r bits is combined with c zero bits. This combined block is then XORed with the current state to generate the input for the f function, where in SHA-3, the f function corresponds to the KECCAK algorithm, which executes 24 rounds of five step mappings theta, rho, pi, chi, and iota [7].
In the squeezing phase, the f function is iteratively applied to generate the hash value with a specified size, which varies depending on the chosen mode, as shown in Fig. 1. For SHA3-224/256/384/512, as the output size d specified in Table I is always less than the 1600 bits output from the absorbing phase, truncation is conducted at this phase without further utilization of the f function.

Table I. The parameters of the SHA-3 modes.

Mode	SHA3-224	SHA3-256	SHA3-384	SHA3-512
Rate r	1152	1088	832	576
Capacity c	448	512	768	1024
Output Size d	224	256	384	512

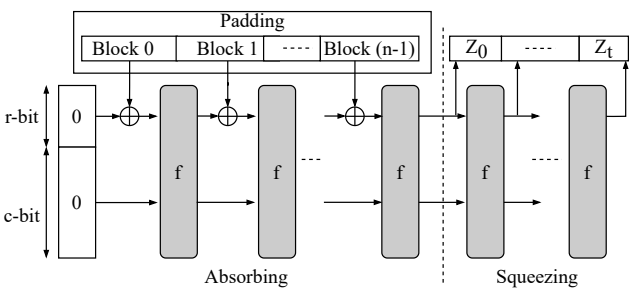


Fig. 1. SPONGE construction.

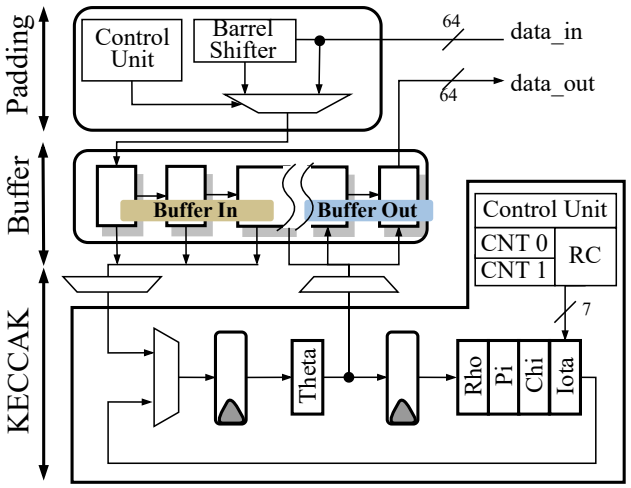


Fig. 2. Overview of proposed MS3 architecture.

3. Proposed Hardware Architecture

This section outlines the proposed hardware-based MS3 architecture, providing an architecture of each internal component in detail. Key components discussed include the configurable buffer, KECCAK, and the integration of MS3 into the RISC-V system.

3.1 Overview of the MS3 architecture

The overview of the proposed MS3 architecture, depicted in Fig. 2, consists of several vital blocks: Buffer, Padding, and KECCAK. The Buffer block, comprising Buffer In (BI) and Buffer Out (BO), manages input and output messages, respectively. Each BI and BO are 64 bits in size. BI receives 64-bit data from external sources, such as memory, and accumulates it to a maximum of 1152 bits or 896 bits for SHA3-224/256 or SHA3-384/512, respectively. Conversely, BO receives input in 512-bit chunks and externally outputs each 64-bit chunk. Padding is performed during data reception before the BI block. The control unit block in the padding phase calculates the input size to determine which 64-bit chunk data requires padding. The KECCAK block includes five primary step mappings: theta, rho, pi, chi, and iota [7]. Each message undergoes 24 rounds before being passed to the BO block.

3.2 Configurable Buffer

Since our MS3 architecture supports multiple modes with varying input and output sizes, it necessitates an appropriate number of buffers to accommodate these differences. In the case of a 64-bit system bus, the SHA3-224 mode requires 18 BIs and 4 BOs, SHA3-256 mode requires 17 BIs and 4 BOs, SHA3-384 mode requires 13 BIs and 6 BOs, and SHA3-512 mode requires 9 BIs and 8 BOs. To optimize buffer usage and reduce complexity, we introduce configurable buffers between the BI and BO sets for certain modes. As depicted in Fig. 3, a total of 22 buffers are allocated for both BI and BO across all modes. To simplify the configuration, we

group the modes into two main categories: Group 1 (G1), consisting of SHA3-384 and SHA3-512 modes, and Group 2 (G2), comprising SHA3-224 and SHA3-256 modes. In G1, the first 14 buffers are designated for BI, while the remaining buffers are reserved for BO. Conversely, in G2, the first 18 buffers are allocated to BI, leaving the rest for BO. However, there are specific buffers (Buffer 14, 15, 16, and 17) that serve dual purposes, acting as both BI in G1 and BO in G2. These buffers, known as Buffer In/Out (BIO), are configured as either BI or BO depending on the mode used.

Fig. 4 illustrates the proposed buffer architecture, with 14 buffers designated as BI. BI 0 receives data from the padding block, and the remaining BIs (Buffers 1 to 13) receive data from the previous buffer when the *valid_in* signal is active. Otherwise, they retain their current values. The four buffers from 18 to 21 are always designated as BO. These buffers receive data from the KECCAK block when the *valid_out* signal is active. Otherwise, they receive data from the previous buffer to shift the data outside. The four middle buffers from 14 to 17 are referred to as BIO. Their functionality depends on the selected mode, determined by the *sel* signal. When *sel* is set to 1, corresponding to SHA3-224/256 mode, these BIOs function as BI; otherwise, they operate as BO. In cases where the input data cannot fill the required number of BI buffers (18/18/14/14 for SH3-224/256/384/512 modes, respectively), zeros are inserted into BI to ensure a sufficient number.

For BI, data is fed to the KECCAK block once enough data fills either the first 18 BIs or the first 14 BIs, depending on SHA3-224/256 or SHA3-384/512, respectively. On the other hand, for BO, the output, which can be up to 512 bits, is fed in parallel into the BOs. In all modes, the least significant bit (LSB) is consistently placed into BO 21. However, the most significant bit (MSB) is placed into BO 18/18/16/14 for SHA3-224/256/384/512 modes, respectively.

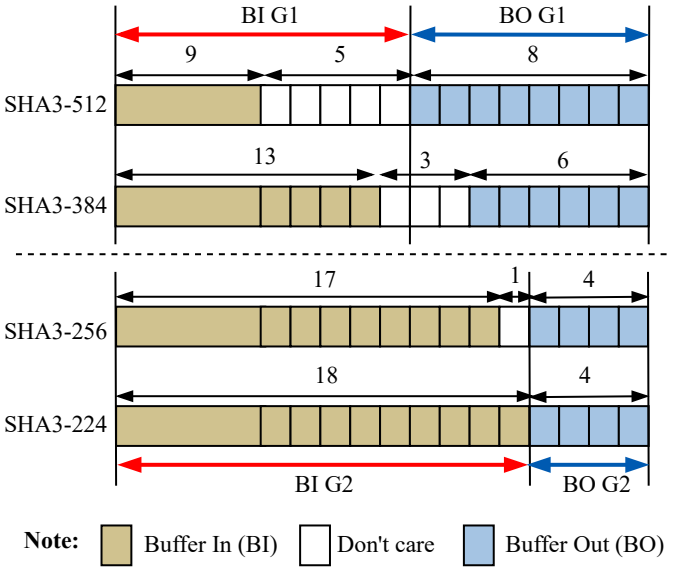


Fig. 3. BI and BO sizes of modes.

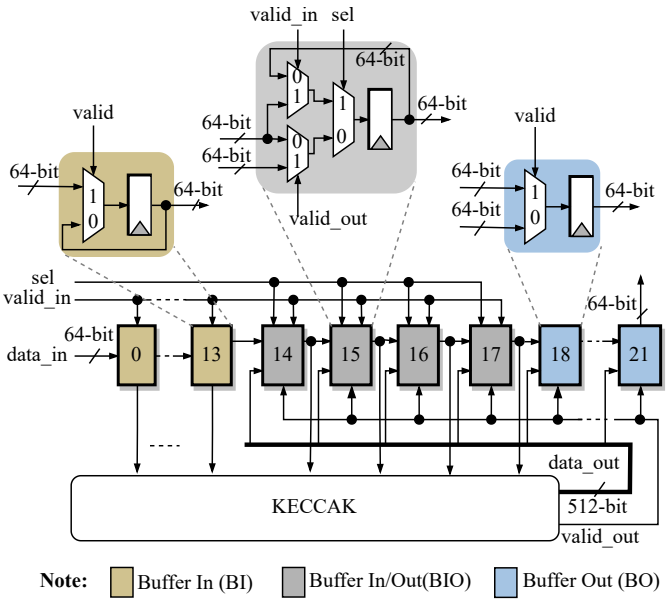


Fig. 4. The configurable buffer architecture.

3.3 Padding

Our padding mechanism utilizes a barrel shifter with a 6-bit shift [25]. Our approach offers resource savings by processing only 64 bits per padding operation. One notable distinction of our padding method is its support for the bit level, in contrast to byte-level padding employed in previous research [13, 14]. Our padding enables more detailed and flexible padding configurations.

3.4 KECCAK

In the KECCAK architecture, we employ the sub-pipeline technique, which involves inserting two registers before and after the theta step mapping, as depicted in Figure 2. This approach reduces the critical path because the theta step mapping incurs more delay than other steps. Additionally, A. Dolmeta et al. in [26] indicated that rearranging the sequence of step mappings can enhance the maximum frequency. Their study highlighted that the theta and chi step mappings offer higher levels of parallelization than others. Consequently, arranging the sequence as chi - iota - theta at the end can yield the best frequency results. This experiment suggests avoiding positioning the theta step mapping at the beginning of the sequence. Therefore, in our approach, we isolate the theta step mapping from the others, thus optimizing the step mapping order to improve performance. For the iota step mapping, we use a lookup table (LUT) to store the 7-bit round constant (RC), following the approach outlined in [15], as illustrated in Fig. 2. This strategy offers a resource-efficient solution compared to storing the full 64-bit constant, as demonstrated in prior works such as [13, 27, 28].

To optimize the utilization of the sub-pipeline KECCAK, we utilize two counters, counter 0 (CNT 0) and counter 1 (CNT 1), to provide two slots (slot 0 and slot 1) for the concurrent processing of two distinct data, as

depicted in Fig. 2. Each counter tracks the number of rounds completed for its corresponding data. Following every two clock cycles, the counters are incremented until a total of 24 rounds are completed. This parallel processing capability enables the hash value generation in an average of 24 clock cycles. Simultaneously processing two data, the architecture efficiently handles multiple inputs while consistently leveraging 100% of the sub-pipeline's resources.

3.5 The MS3 accelerator based on the RISC-V system

MS3 is integrated into RISC-V system as the hardware accelerator shown in Fig. 5. RISC-V core named Rocket with RV64IMAC instruction set architecture (ISA) extensions [29] is connected with other components through TileLink Interconnection such as system bus (SBus), memory bus (MBus), peripheral bus (PBus), and front bus (FBus) [30]. PBus comprises various peripheral components including the boot read-only memory (ROM), which contains the initial program required for system boot-up, the serial peripheral interface (SPI) responsible for interfacing with external storage modules such as secure digital (SD) card which contains the software for the system, and the universal asynchronous receiver-transmitter (UART) enabling communication with a personal computer (PC) for display purposes. Additionally, the MS3 CFG block receives a configuration from the Rocket core to set up the MS3 accelerator for operation. FBus is the master interface for DMA operations [30]. Upon receiving the necessary configuration data, such as read address, write address, data length, and start signal, the MS3 CFG block triggers the Read Master block to request data from random-access memory (RAM) in MBus, thanks to DMA. This data is stored in a first in first out (FIFO). Once the FIFO contains the requisite data, the MS3 accelerator starts operation. Upon completion, the generated hash value is outputted to the FIFO, where the Write Master promptly transfers it to the specified write address in RAM.

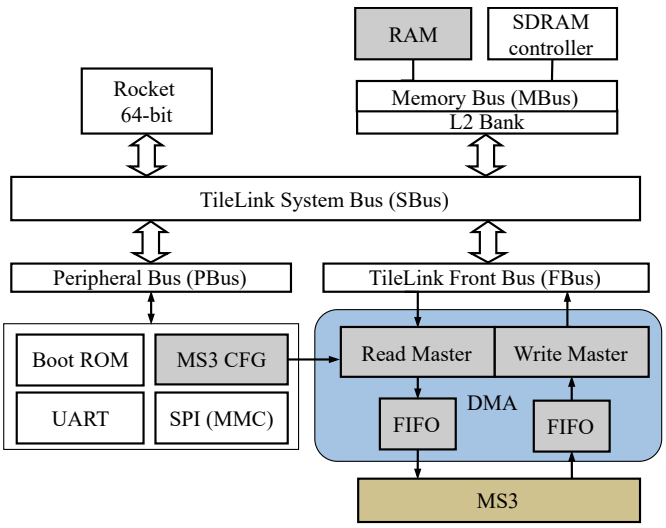


Fig. 5. The MS3 accelerator based on RISC-V system.

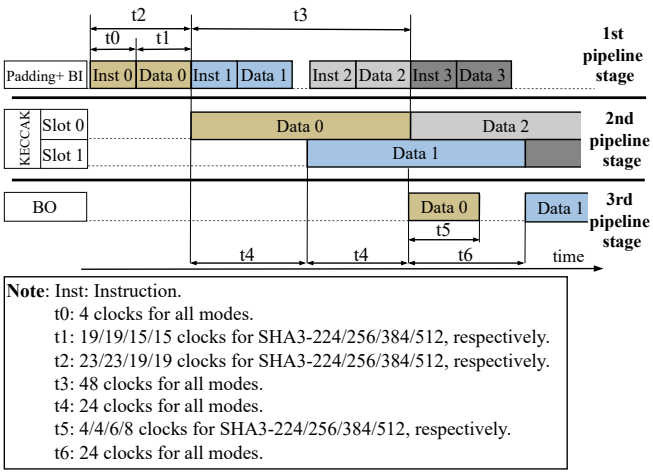


Fig. 6. Timing chart of MS3 on the system level.

Fig. 6 illustrates the timing chart of the MS3 accelerator on the RISC-V system. Our MS3 accelerator operates using a pipeline mechanism at the system level, comprising three states: Padding + BI, KECCAK, and BO.

During $t2$ (23/23/19/19 clocks for SHA3-224/256/384/512 modes, respectively), Padding and BI blocks receive data from the FIFO. $t0$ accounts for 4 clocks for handling and controlling instructions, including mode and data length for the current handled data. $t1$ requires 18/18/14/14 clocks to receive data from the FIFO for SHA3-224/256/384/512 modes, respectively, and an additional clock for the done signal, totaling 19/19/15/15 clocks for SHA3-224/256/384/512 modes, respectively. Therefore, the 1st pipelined stage needs at least $t2$.

Subsequently, this chunk of data is fed into KECCAK. The processing time for one data chunk in KECCAK is $t3$ (48 clocks for all modes). With two slots available to handle two different data chunks, KECCAK can process continuous data in an average of 24 clocks. Thus, the input interval of KECCAK is $t4$ (24 clocks for all modes).

The output of the KECCAK function is then transferred to BOs. BOs require $t5$ (4/4/6/8 clocks for SHA3-224/256/384/512 modes, respectively) to complete their processing. The time interval required to generate one hash value is denoted as $t6$, which is 24 clocks for all modes because the 2nd pipelined stage takes the most time for processing, requiring 24 clocks.

Table II. Synthesis results on Cyclone IV E.

Design		MS3	RISC-V system
Frequency (MHz)		220	50
Area	LEs *	7402 (6%)	31761 (28%)
	Registers	4752	16972
	DSP †	0	20
	Memory bits	0 (0%)	330432 (8%)

* LE: Logic element
† DSP: Digital signal processor

Table III. The synthesis results of KECCAK on Xilinx FPGA.

Reference		[13]		[20]		[14]			This work		
Approach		Unrolling k=12		Dual Rnd		Unrolling k=2			Sub-pipeline		
FPGA device		Virtex 5	Virtex 6	Virtex 5	Virtex 7	Virtex 5	Virtex 6	Virtex 7	Virtex 5	Virtex 6	Virtex 7
Mode	SHA3-224	✔		✖		✔			✔		
	SHA3-256	✔		✖		✔			✔		
	SHA3-384	✔		✖		✔			✔		
	SHA3-512	✔		✔		✔			✔		
Fmax (MHz)		41.64	57.91	-	-	203.28	347.84	378.73	461.32	605.11	720.46
Area (Slice)		12487	15579	1375	1521	1112	1287	1375	1074	966	940
Tp. (Gbps)*		11.99	16.67	17.67	22.90	9.76	16.69	18.18	11.07	14.52	17.29
Eff. (Mbps/Slice) *		0.96	1.07	12.85	15.11	8.77	12.97	13.22	10.31	15.03	18.39

* For SHA3-512 mode

4. Results

The proposed architecture, integrating MS3 with the RISC-V system, has been implemented and tested on the Altera DE2-115 Development and Education Board (Cyclone IV EP4CE115). Random samples (1K Byte) were created for testing each mode, and MS3's throughput was measured using Eq. (1), where one counter is used to track the number of elapsed clocks ($\#num_clock$). The system operates at 50 MHz, while MS3 operates at 220 MHz, as shown in Table II. As a result, DMA achieves a throughput of 540.21 Mbps while MS3's throughput is 520.46/497.52/495.62/478.81 Mbps for SHA3-224/256/384/512 modes, respectively. The throughput of DMA constrains the throughput of MS3. As shown in Table II, resource utilization indicates that the RISC-V system occupies 28% of the field programmable gate arrays (FPGA) logic. In contrast, MS3 only occupies 6% without utilizing any digital signal processor (DSP) or memory bits.

Moreover, efficiency was calculated as the ratio of throughput to resource utilization, following the formula described in Eq. (2). To simplify the comparison and evaluation process, only the results of the SHA3-512 mode are presented, as all modes share the common KECCAK. Consequently, variations in throughput and efficiency are based on the number of input bits ($\#num_bit$).

$$Throughput = \frac{\#num_bit \times frequency}{\#num_clock} \quad (1)$$

$$Efficiency = \frac{Throughput}{Area} \quad (2)$$

The synthesis results on Xilinx FPGA devices (Virtex-5 5VLX50T, Virtex-6 6VLX75T, and Virtex-7 XC7VX485T), as presented in Table III, allow for comparison and evaluation with other works [13, 14, 20].

In [13], S. El Moumni et al. proposed an unrolling approach with multiple factors k ranging from 2 to 12. Despite achieving the best throughput with k=12, their design exhibited poor efficiency due to increased resource utilization aimed at reducing the number of clocks. In contrast, our implementation of KECCAK utilizes the sub-pipeline technique, which enhances throughput while maintaining low resource utilization. This technique reduces half of the critical path by inserting extra registers but keeps the operated clock at 24 using two counters which require few resources. Specifically, our design achieves efficiencies 10.74 (10.31 vs. 0.96) and 14.05 (15.03 vs. 1.07) times higher and throughputs 0.92 (11.07 vs. 11.99) and 0.87 (14.52 vs. 16.67) times higher compared to [13], on Virtex 5 and 6, respectively.

M. M. Sravani et al. [20] propose another architecture to boost throughput. The dual Rnd approach in [20] utilizes two Rnd blocks and two 1600-bit registers per round, resulting in a higher slice count than our design. Consequently, our efficiency is 1.22 (18.39 vs. 15.11) times higher on Virtex 7. Moreover, our design supports multiple modes (SHA3-224/256/384/512).

In [14], A. Sideris et al. reduced the required clock cycles to 12 by unrolling with k=2 to improve throughput. In contrast, our design employs the sub-pipeline technique, boosting frequency and throughput while maintaining a consistent clock cycle count (24 clocks). Although both approaches achieve similar throughputs on Virtex 5, 6, and 7, our design maintains high throughput with lower slice utilization. As a result, our efficiency is 1.17 (10.31 vs. 8.77), 1.16 (15.03 vs. 12.97), and 1.39 (18.39 vs. 13.22) times higher compared to [14] on Virtex 5, 6, and 7, respectively.

5. Conclusion

Flexible and efficient hardware architectures to support various SHA-3 modes are useful for many real applications. This paper introduces an efficient MS3 architecture with configurable buffers to save resources and a sub-pipeline approach involving register insertion before and after the theta step mapping to enhance throughput. Leveraging the open-source RISC-V ISA, particularly the Rocket RV64IMAC ISA offers the opportunity to implement the design in an application-specific integrated circuit (ASIC). Integrating MS3 into the Rocket system as an accelerator and using DMA enhances dataflow throughput. The proposed architecture is implemented and tested on

Cyclone IV E, MS3 achieves a frequency of 220 MHz, with a throughput of 520.46/497.52/495.62/478.81 Mbps for SHA3-224/256/384/512 modes, respectively. Synthesis results on Virtex 5, 6, and 7 reveal efficiencies of 10.31, 15.03, and 18.39 Mbps/Slice for the case of SHA3-512 are better than previous works, respectively.

Hardware security has become a recent trend, ranging from FPGA to ASIC implementations. Our current design is exclusively implemented on FPGA devices. Thus, leveraging the open-source nature of RISC-V, our future work involves implementing the entire MS3 and RISC-V system on ASIC platforms.

Acknowledgments

This research is funded by University of Science, VNU-HCM under grant number ĐT-VT 2024-03.

References

[1] S. Shin, *et al.*: "A Privacy-preserving Authentication, Authorization, and Key Agreement Scheme for Wireless Sensor Networks in 5G-integrated Internet of Things," IEEE access **8** (2020) 67555 (DOI: 10.1109/ACCESS.2020.2985719).

[2] S. Jiang, *et al.*: "An Efficient Anonymous Batch Authentication Scheme Based on HMAC for VANETs," IEEE Trans. on Intelligent Transportation Syst. **17** (2016) 2193 (DOI: 10.1109/TITS.2016.2517603).

[3] L. Zhou, *et al.*: "A Lightweight Cryptographic Protocol with Certificateless Signature for the Internet of Things," ACM Trans. on Embedded Comp. Syst. (TECS) **18** (2019) 1 (DOI: 10.1145/3301306).

[4] K. A. Nugroho, *et al.*: "SHA-2 and SHA-3 Based Sequence Randomization Algorithm," Int. Conf. on Science and Tech.-Comp. (ICST) (2016) 150 (DOI: 10.1109/ICSTC.2016.7877365).

[5] A. Sharma, *et al.*: "Cryptography & Network Security Hash Function Applications, Attacks and Advances: A Review," Int. Conf. on Inventive Syst. and Control (ICISC) (2019) 177 (DOI: 10.1109/ICISC44355.2019.9036448).

[6] W. Wang, *et al.*: "A Survey on Consensus Mechanisms and Mining Strategy Management in Blockchain Networks," IEEE Access **7** (2019) 22328 (DOI: 10.1109/ACCESS.2019.2896108).

[7] Federal Information Processing Standards Publication: "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," (2015) DOI: 10.6028/NIST.FIPS.202.

[8] M. Stevens, *et al.*: "The First Collision for Full SHA-1," Annual Int. Crypto. Conf. Advances in Crypto. (CRYPTO) (2017) 570 (DOI: 10.1007/978-3-319-63688-7_19).

[9] S. K. Sanadhya, *et al.*: "New Collision Attacks Against Up to 24-Step SHA-2," Progress in Cryptology (INDOCRYPT) 2008 (2008) 91 (DOI: 10.1007/978-3-540-89754-5_8).

[10] Federal Information Processing Standards Publication: "Secure Hash Standard (SHS)," 2015 DOI: 10.6028/NIST.FIPS.180-4.

[11] M. Sundal, *et al.*: "Efficient FPGA Implementation of the SHA-3 Hash Function," 2017 IEEE Comp. Society Annual Symp. on VLSI (ISVLSI) (2017) 86 (DOI: 10.1109/ISVLSI.2017.24).

[12] B. Jungk, *et al.*: "Serialized Lightweight SHA-3 FPGA Implementations," Microprocessors and Microsystems **71** (2019) 102857 (DOI: 10.1016/j.micpro.2019.102857).

[13] S. El Moumni, *et al.*: "High Throughput Implementation of SHA3 Hash Algorithm on Field Programmable Gate Array (FPGA)," Microelec. J. **93** (2019) 104615 (DOI: 10.1016/j.mejo.2019.104615).

[14] A. Sideris, *et al.*: "Hardware Acceleration Design of the SHA-3 for High Throughput and Low Area on FPGA," J. of Crypto. Engi. (2023) 1 (DOI: 10.1007/s13389-023-00334-0).

[15] A. Sideris, *et al.*: "A Novel Hardware Architecture for Enhancing the Keccak Hash Function in FPGA Devices," Info. **14** (2023) 475 (DOI: 10.3390/info14090475).

[16] M. M. Wong, *et al.*: "A New High Throughput and Area Efficient SHA-3 Implementation," IEEE Int. Symp. on Circ. and Syst. (ISCAS) (2018) 1 (DOI: 10.1109/ISCAS.2018.8351649).

[17] H. Mestiri, *et al.*: "High Throughput Pipelined Hardware Implementation of the KECCAK Hash Function," Int. Symp. on Signal, Image, Video and Comm. (ISIVC) (2016) 282 (DOI: 10.1109/ISIVC.2016.7894001).

[18] H. E. Michail, *et al.*: "Pipelined SHA-3 Implementations on FPGA: Architecture and Performance Analysis," Workshop on Crypto. and Secu. in Comp. Syst. (2015) 13 (DOI: 10.1145/2694805.2694808).

[19] F. Assad, *et al.*: "High-performance FPGA Implementation of the Secure Hash Algorithm 3 for Single and Multi-message Processing," Int. J. of Elec. and Comp. Engi. (IJECE) **12** (2022) 1324 (DOI: 10.11591/ijece.v12i2.pp1324-1333).

[20] M. M. Sravani, *et al.*: "On Efficiency Enhancement of SHA-3 for FPGA-based Multimodal Biometric Authentication," IEEE Trans. on Very Large Scale Integration (VLSI) Syst. **30** (2022) 488 (DOI: 10.1109/TVLSI.2022.3148275).

[21] W. Wang, *et al.*: "An Energy-efficient Crypto-extension Design for RISC-V," Microelec. J. **115** (2021) 105165 (DOI: 10.1016/j.mejo.2021.105165).

[22] J. Rao, *et al.*: "Design Exploration of SHA-3 ASIP for IoT on a 32-bit RISC-V Processor," IEICE Trans. on Info. and Syst. **101** (2018) 2698 (DOI: 10.1587/transinf.2017ICP0019).

[23] T. T. Hoang, *et al.*: "Quick Boot of Trusted Execution Environment with Hardware Accelerators," IEEE Access **8** (2020) 74015 (DOI: 10.1109/ACCESS.2020.2987617).

[24] B. Peccerillo, *et al.*: "A Survey on Hardware Accelerators: Taxonomy, Trends, Challenges, and Perspectives," J. of Syst. Arch. **129** (2022) 102561 (DOI: 10.1016/j.sysarc.2022.102561).

[25] A. Ito: "Barrel shifter," US Patent 4,829,460. May 1989.

[26] A. Dolmeta, *et al.*: "Comparative Study of Keccak SHA-3 Implementations," Crypto. **7** (2023) 60 (DOI: 10.3390/cryptography7040060).

[27] S. El Moumni, *et al.*: "High Frequency Implementation of Cryptographic Hash Function Keccak-512 on FPGA Devices," Int. J. of Info. and Comp. Secu. **10** (2018) (DOI: 10.1504/IJICS.2018.095299).

[28] A. Sideris, *et al.*: "High Throughput Implementation of the Keccak Hash Function Using the Nios-II Processor," Tech. **8** (2020) 15 (DOI: 10.3390/technologies8010015).

[29] A. Waterman, *et al.*: "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.0," EECS Department, Univ. of California, Berkeley (2014) http://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-54.html.

[30] A. Amid, *et al.*: "Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs," IEEE Micro **40** (2020) 10 (DOI: 10.1109/MM.2020.2996616).